26F Stat TA Session W3

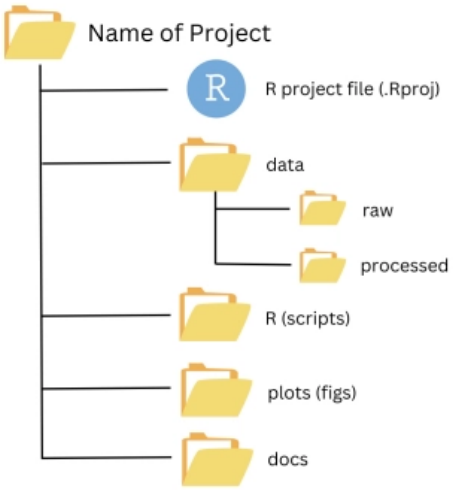
0. Preparation for TA session

- Please download the W3 example code and the attached dataset click here
  - This week we'll use the dataset `loan_full_schema` , which is a extended version of `loan50` (the W2 dataset)
- Import `loan_full_schema` and `county`

1. (Non-)Trivial things

Folder Organization

- Although it sounds simple, but it's important to have well-organized folders for your working project!
- Different types of files (code scripts, datasets, output, etc.) should be saved in different folders.



Coding Styles

- You can write your code in any ways as long as it's runnable. But it's important to allow others to read your codes and understand what you're doing!
  - Check this website to learn more about readable codes

> Tidyverse style guide
>
> Good coding style is like correct punctuation: you can manage without it, butitsuremakesthingseasiertoread. This site describes the style used throughout the tidyverse. It was derived from Google's original R Style Guide - but Google's
>
> https://style.tidyverse.org/

- **Naming:** Only use lowercase letters, numbers, and use `_` to replace any spaces

```
# Good lesson_one lesson_1 # Not good LessonOne lessonone # Disaster LeSSoNone one a <-
1
```

- **Spacing:**
  - Always put a space after a comma `,` and never put one before it
  - Do not put spaces inside or outside parentheses (括號)
  - You may put spaces before and after `=`

```
# Good x[, 1] mean(x, na.rm = TRUE) # Bad x[,1] x[ ,1] x[ , 1] mean (x, na.rm =
TRUE) mean( x, na.rm = TRUE )
```

- **Function Indents (縮排):**
  - Indent the argument name with a single indent
  - Avoid putting `+` or `,` at the head of a new line!

```
# Good long_function_name <- function( a = "a long argument", b = "another
argument", c = "another long argument" ) { } # Bad long_function_name <- function(a
= "a long argument", b = "another argument", c = "another long argument" ) { }
long_function_name <- function(a = "a long argument", b = "another argument", c =
"another long argument"){}
```

Clean up the environment

- When dealing with multiple datasets at the same time, you may end up with too many variables in the global environment. e.g., variables of previous analyses
- You can use `rm(list = ls())` to clean up the workspace, especially when you want to rerun the entire code
  - `ls()` returns a character vector of object names in the current environment

```
x <- 1 y <- "hi" ls() # [1] "x" "y"
```

2. Summary Statistics

Create a contingency table `table()`

```
# dnn = dimension names A <- table(..., dnn = ...) # example loan <-
read.csv("loans_full_schema.csv") table <- table(loan$application_type,
loan$homeownership, dnn = c("application type", "home ownership" ) ) # output home
ownership application type MORTGAGE OWN RENT individual 3839 1170 3496 joint 950 183 362
```

- `table()` gives us numbers for each category
- We can also count the total number for each category along using `addmargins`

```
table_withsum <- addmargins(table) # output home ownership application type MORTGAGE OWN
RENT Sum individual 3839 1170 3496 8505 joint 950 183 362 1495 Sum 4789 1353 3858 10000
```

- `addmargins()` adds marginal totals to the original table
  - We can also use other functions to extend the table

```
# example: minimal values addmargins(table, FUN = list(list(MIN = min))) # output
Margins computed over dimensions in the following order: 1: application type 2:
home ownership home ownership application type MORTGAGE OWN RENT MIN individual
3839 1170 3496 1170 joint 950 183 362 183 MIN 950 183 362 183
```

3. More data visualization

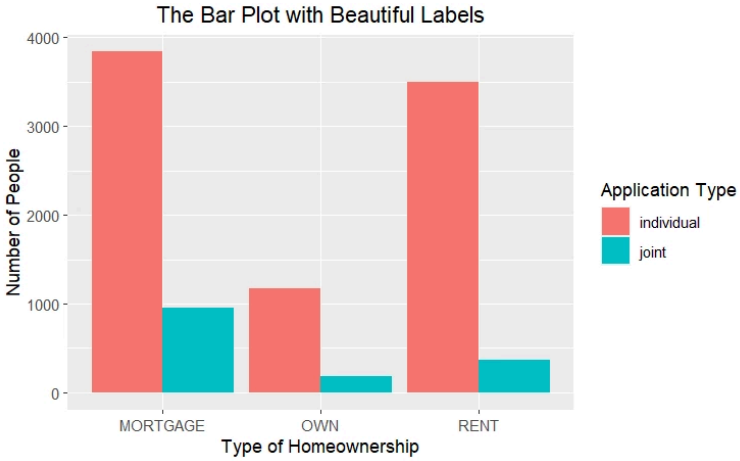
Bar plot `geom_bar()`

```
ggplot(loan, aes(x = homeownership, fill = application_type)) + geom_bar(position =
"dodge") # the bars dodge each other
```

Figure: Bar plot of homeownership by application type

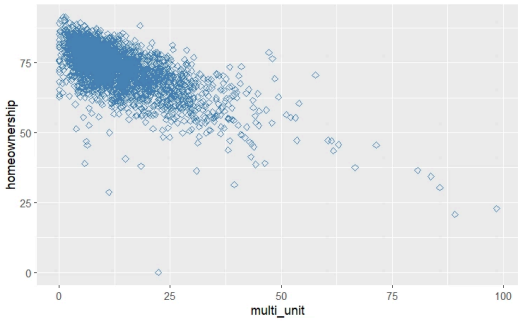
- We can add title and axis labels to improve the readability of the figure

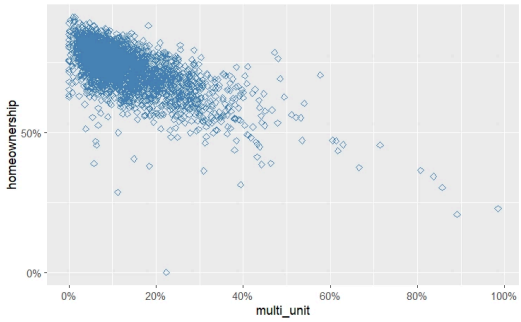
```
your_plot + labs(x = ..., y = ..., title = ...) + theme(plot.title = element_text(hjust = 0.5)) # example bar_graph + labs(x = "Type of Homeownership", y = "Number of People", title = "The Bar Plot with Beautiful Labels", fill = "Application Type" ) + theme(plot.title = element_text(hjust = 0.5)) # center the title
```



- Another trick is to adjust the scale of x and y-axes
  - Again we can relabel the scales!

```
your_plot + scale_x_continuous(breaks = ..., labels = ...) + scale_y_continuous(breaks = ..., labels = ...) # example # recall scatter plot we learned last week scatter_plot <- ggplot(county, aes(x = multi_unit, y = homeownership)) + geom_point(color = "steelblue", shape = 5) scatter_plot_scale <- scatter_plot + scale_x_continuous( breaks = c(0, 20, 40, 60, 80, 100), labels = c("0%", "20%", "40%", "60%", "80%", "100%") ) + scale_y_continuous( breaks = c(0, 50, 100), labels = c("0%", "50%", "100%") )
```





▶ **Q. What's the difference between a histogram and a bar plot?**

💡 Always check if your data visualization strategy is valid and appropriate!

Exercises

W3 Exercises